```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
text = """
artificial intelligence is transforming modern society.
it is used in healthcare finance education and transportation.
machine learning allows systems to improve automatically with experience.
data plays a critical role in training intelligent systems.
large datasets help models learn complex patterns.
deep learning uses multi layer neural networks.
neural networks are inspired by biological neurons.
each neuron processes input and produces an output.
training a neural network requires optimization techniques.
gradient descent minimizes the loss function.
natural language processing helps computers understand human language.
text generation is a key task in nlp.
language models predict the next word or character.
recurrent neural networks handle sequential data.
lstm and gru models address long term dependency problems.
"""
```

```
text = text.lower().replace(".", "")
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])

word_index = tokenizer.word_index
index_word = tokenizer.index_word
vocab_size = len(word_index) + 1
```

```
tokens = tokenizer.texts_to_sequences([text])[0]

SEQ_LENGTH = 5
X = []
y = []

for i in range(len(tokens) - SEQ_LENGTH):
    X.append(tokens[i:i + SEQ_LENGTH])
    y.append(tokens[i + SEQ_LENGTH])

X = np.array(X)
y = np.array(y)
```

```
model = Sequential([
    Embedding(input_dim=vocab_size, output_dim=64, input_length=SEQ_LENGTH),
    LSTM(128),
    Dense(vocab_size, activation="softmax")
])

model.compile(
    loss="sparse_categorical_crossentropy",
    optimizer="adam"
)

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:97: UserWarning: Argument `input_length` is depre
  warnings.warn(
```

Model: "sequential_2"

Layer (type)	Output Shape	Param #
embedding_2 (Embedding)	?	0 (unbuilt)
lstm_2 (LSTM)	?	0 (unbuilt)
dense_2 (Dense)	?	0 (unbuilt)

Total params: 0 (0.00 B)
Trainable params: 0 (0.00 B)

```
model.fit(
    X,
    y,
    epochs=40,
    batch_size=16
)
```

```
Epoch 1/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 4.5337
Epoch 2/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 4.5222
Epoch 3/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 4.5128
Epoch 4/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 4.5025
Epoch 5/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 4.4870
Epoch 6/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 4.4648
Epoch 7/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 4.4350
Epoch 8/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 4.3689
Epoch 9/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 4.2516
Epoch 10/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 4.0349
Epoch 11/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 3.8650
Epoch 12/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.6433
Epoch 13/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 3.3288
Epoch 14/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 2.8427
Epoch 15/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 2.6636
Epoch 16/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 2.3331
Epoch 17/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.9561
Epoch 18/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.7752
Epoch 19/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.5055
Epoch 20/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 1.3312
Epoch 21/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 1.1282
Epoch 22/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - loss: 1.0459
Epoch 23/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.9357
Epoch 24/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.8310
Epoch 25/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - loss: 0.7125
Epoch 26/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.6379
Epoch 27/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.5461
Epoch 28/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - loss: 0.4816
Epoch 29/40
7/7 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 0.4388
```

```
def generate_text(seed_text, length=30):
    seed_text = seed_text.lower().split()
    generated = seed_text.copy()

    for _ in range(length):
        encoded = tokenizer.texts_to_sequences(
            [" ".join(generated[-SEQ_LENGTH:])]
        )[0]

        encoded = pad_sequences(
            [encoded], maxlen=SEQ_LENGTH, padding="pre"
        )

        prediction = model.predict(encoded, verbose=0)
        next_word_index = np.argmax(prediction)
        next_word = index_word.get(next_word_index, "")

        generated.append(next_word)
```

```
    return " ".join(generated)
```

```
print(generate_text("artificial intelligence is"))
```

```
artificial intelligence is networks society society it healthcare a in healthcare finance transportation nlp transportation
```